

AgriEasy — Interview Preparation Guide

For: Rishabh Gupta | Full-Stack Development Interview

PART 1: AGRIEASY PROJECT DEEP DIVE

Project Overview (30-second pitch)

Architecture

Database Models (18)

Authentication Flow

API Design

Frontend Architecture

Key Features Deep Dive

Thought Process: How I Built It

PART 2: INTERVIEW QUESTIONS

Basic Full-Stack Questions

Intermediate Questions

Advanced Questions

System Design Questions

Behavioral / Project-Specific Questions

PART 3: QUICK REFERENCE

Numbers to Remember

Key Technologies to Mention

If Asked “Tell me about your project”

AgriEasy — Interview Preparation Guide

For: Rishabh Gupta | Full-Stack Development Interview

PART 1: AGRIEASY PROJECT DEEP DIVE

Project Overview (30-second pitch)

“AgriEasy is a three-sided agricultural marketplace PWA connecting Farmers, Buyers, and Transporters. Built with Next.js 16, TypeScript, and MongoDB, it features 72 REST API endpoints across 55 pages, with JWT authentication,

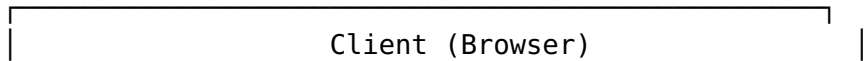
Razorpay payments, real-time social feed, bill OCR, and PWA offline support. The app handles the entire supply chain — from a farmer listing produce, to a buyer placing an order, a transporter delivering it, and payment settlement.”

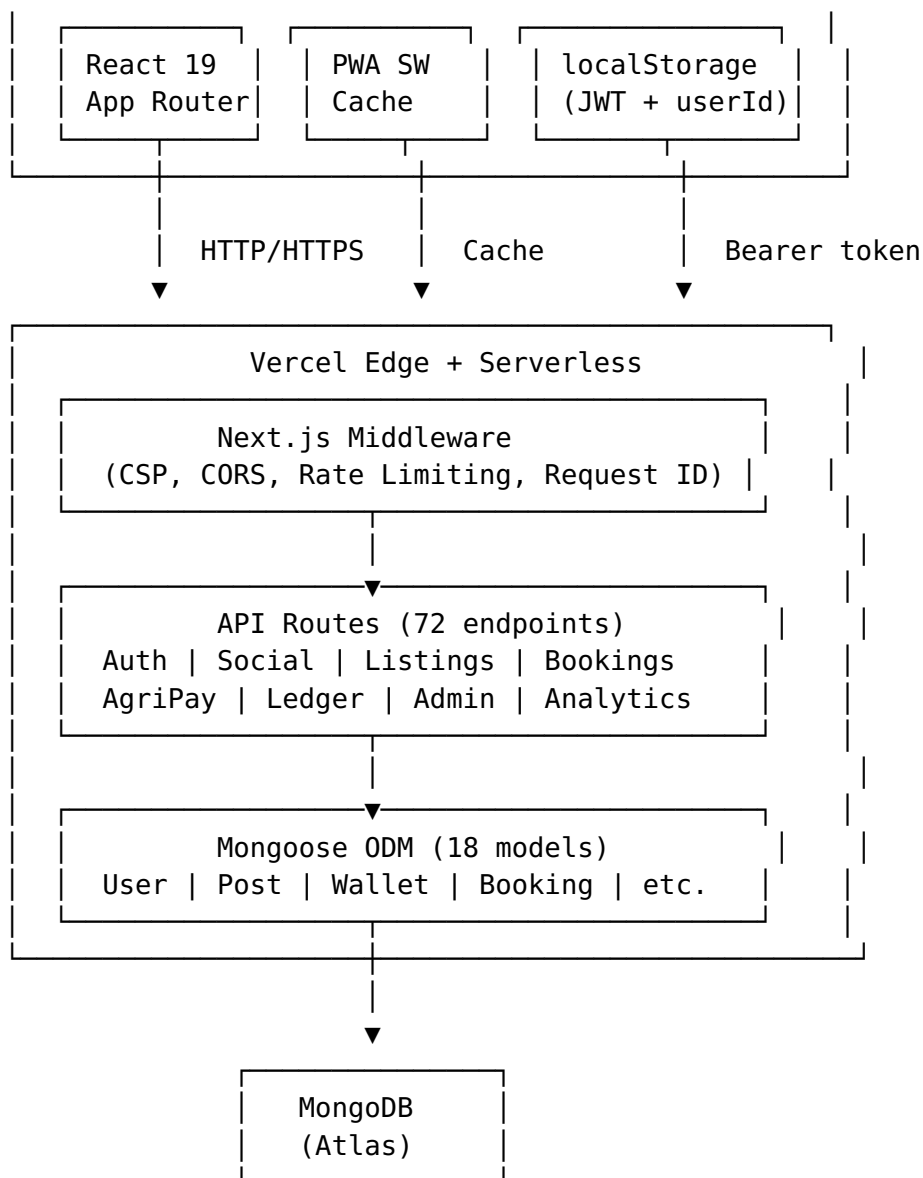
Architecture

Tech Stack

Layer	Technology
Framework	Next.js 16 (App Router, Turbopack)
Language	TypeScript (strict mode)
Frontend	React 19, inline CSS-in-JS styles, custom design system
Backend	Next.js API Routes (serverless functions on Vercel)
Database	MongoDB + Mongoose 9 (18 models)
Auth	JWT (7-day expiry) + NextAuth.js (Google OAuth, disabled for verification)
Payments	Razorpay (wallet topups, booking payments)
File Storage	Cloudinary (signed uploads for social media + bill photos)
OCR	Qwen3-VL via OpenRouter (bill calculator) + Tesseract.js fallback
PWA	Service Worker (stale-while-revalidate caching), Web App Manifest
Security	CSP headers, HSTS, bcrypt password hashing, AES-256-GCM encryption for PII
Deployment	Vercel (auto-deploy on push to main)
CI/CD	Cloudflare Workers Builds (OCR proxy worker)

Architecture Pattern





Key Numbers

- **72 API endpoints** (44 GET, 51 POST, 9 PATCH, 6 DELETE)
- **55 pages** (48 static, 9 dynamic with [id] params)
- **18 Mongoose models**
- **~28,700 lines** of TypeScript/TSX code
- **3 user roles:** Farmer/Vyapari, Buyer, Transporter
- **Service Worker** with stale-while-revalidate caching (v25)

Database Models (18)

Model	Purpose	Key Fields
User	User accounts (3 roles)	email, phone, password (bcrypt), role, address,

Model	Purpose	Key Fields
		aadharNumber (AES-256 encrypted), tokenVersion
Post	Social feed posts + KrishiClips	type (post/ krishiclip), caption, mediaUrls, likes[], viewedBy[], rankScore
Story	Instagram-style stories	mediaUrl, mediaType, duration, viewedBy[]
Follow	Follow relationships	followerId, followingId
Notification	Push notifications	userId, actorId, type, postId, read
Conversation	DM conversations	participantIds[], lastMessage
Listing	Buyer commodity listings	buyerId, commodity, pricePerUnit, unit, quantity
Booking	Order bookings	listingId, buyerId, transporterId, quantity, status, deliveryAddress
Vehicle	Transporter vehicles	vehicleType, registrationNumber, capacity, baseRatePerKm
BuyerVehicle	Buyer-owned vehicles	vehicleType, freightType, freightAmount
Wallet	AgriPay wallet	userId, balance, agripayId, bankVerified, paylaterEligible
Transaction	Payment records	fromUserId, toUserId, amount, type, status, paymentMethod
PayLater	Credit loans	userId, loanAmount, amountDue, dueDate, status
Ledger	Bill/invoice records	

Model	Purpose	Key Fields
		userId, type, amount, status, counterpartyId, commodities[]
Billing	Buyer billing records	buyerId, amount, status, paymentMode
Collection	Social post collections	userId, name, postIds[]
Highlight	Story highlights	userId, title, storyIds[]
AuditLog	Security audit trail	userId, action, resource, resourceId, details, ipAddress

Authentication Flow

Manual Registration + Login

1. User visits `/auth/register` → fills form (name, email, phone, password, role, address)
2. Zod schema validates input (password regex: uppercase + lowercase + digit + 8+ chars)
3. API hashes password with bcrypt (10 rounds)
4. Aadhar number encrypted with AES-256-GCM before storing
5. User redirected to `/auth/login`
6. Login: email OR phone + password → `bcrypt.compare` → JWT signed (7-day expiry)
7. JWT stored in `localStorage` + `httpOnly` cookie (`sameSite: strict`)
8. `authFetch()` helper adds `Authorization: Bearer <token>` to all API calls

Google OAuth (currently disabled for verification)

1. User clicks "Sign up with Google" → NextAuth.js OAuth flow
2. AuthSync component bridges NextAuth session → JWT → `localStorage`
3. One-click signup: role + phone stored in `sessionStorage` before OAuth
4. On return, AuthSync auto-registers the user via `/api/auth/register`

Security Measures

- JWT with tokenVersion field for revocation (logout increments it)
 - Rate limiting: 3 registrations/min, 5 logins/min, 30 likes/min
 - CSP headers (script-src, img-src, connect-src whitelists)
 - HSTS (max-age=63072000; includeSubDomains; preload)
 - Permissions-Policy (camera, microphone, geolocation = self)
 - XSS protection: xss package strips ALL HTML from user input
 - Zod schema validation on every API endpoint
 - No .passthrough() on Zod schemas (extra fields dropped)
-

API Design

Endpoint Categories

Category	Endpoints	Auth Required
Auth	7 (register, login, logout, OTP, session-token, NextAuth)	Mixed
Social	25 (posts, clips, stories, DM, follow, like, comment, save)	Most require auth
AgriPay	14 (wallet, transfer, topup, history, paylater, verify-bank)	All require auth
Admin	9 (stats, users, posts, wallets, transactions, audit-logs)	Admin role only
Bookings	5 (create, update, pay, confirm-upi, bill)	All require auth
Ledger	4 (CRUD, bill-calc, bill-ocr, bill-calc-proxy)	All require auth
Listings	2 (create, get)	POST requires auth
Vehicles	1	GET public, POST auth
Buyer-vehicles	2	All require auth
Profiles	2 (buyer, farmer)	All require auth
Billing/Payment	4	All require auth

Category	Endpoints	Auth Required
Health/Analytics/ Location	3	Mixed

API Response Format

```
// Success
{ success: true, data: {...}, meta: { page, limit, total,
  totalPages } }

// Error
{ success: false, error: { code: "AUTH_REQUIRED", message:
  "Authentication required" } }
```

Rate Limiting

- In-memory (Map-based) when no Redis configured
- Upstash Redis when available (production)
- Config: { windowMs: 60_000, max: 3 } for registration
- Returns 429 with Retry-After header

Frontend Architecture

Design System

- Custom inline CSS-in-JS (no Tailwind on home page, Tailwind on some internal pages)
- Color palette: Navy (#31372B), Olive (#4A5240), Peach (#E98074), Cream (#FAF7EE)
- Typography: Poppins (5 weights) + Dancing Script (2 weights) via next/font/google
- display: swap to prevent FOIT (Flash of Invisible Text)
- Font weight optimization: only load weights actually used

Key Components

Component	Purpose
BottomTabBar	Role-aware bottom navigation (farmer/buyer/transporter)
PWABootstrap	Service worker registration + install prompt
CookieConsent	GDPR cookie banner (hidden on immersive pages)
AuthSync	Bridges NextAuth session → localStorage JWT

Component	Purpose
FeatureCards	Home page feature cards with “Coming Soon” badges
HomeOpenSound	Cinematic “ta-dum” sound on first click (Web Audio API)

State Management

- No Redux/Zustand — React `useState` + `useCallback` + `useEffect`
- `authFetch()` helper wraps `fetch()` with JWT Bearer token
- Stale-while-revalidate pattern for social feed (localStorage cache → background refresh)
- `sessionStorage` for one-time data (Google signup pending registration)

Performance Optimizations

- Service Worker: network-first for navigations, stale-while-revalidate for static assets
- Lazy loading: images use `next/image` with AVIF/WebP format
- Font weights trimmed (7→5 for Poppins, 4→2 for Dancing Script)
- PWA install prompt: shows immediately (0ms delay)
- `SessionProvider`: `refetchOnWindowFocus={false}` to avoid unnecessary API calls
- Social feed: localStorage cache for 0s first paint on return visits

Key Features Deep Dive

1. AgriSocial Feed (Instagram-style)

- 3 feed modes: following, ranked, latest
- Ranked feed uses $\text{rankScore} = (\text{likes} \times 5 + \text{comments} \times 8 + \text{saves} \times 6 + \text{shares} \times 10) \times \text{recency_decay}$
- `KrishiClips`: vertical scroll video feed with unique view tracking
- View deduplication: `$addToSet` on `viewedBy[]` array (1 view per user per clip)
- Stale-while-revalidate: localStorage cache shows instantly, fresh data swaps in
- Safe populate: `safePopulate()` helper catches corrupt user refs gracefully

2. Bill Calculator (OCR)

- Upload bill photo → Qwen3-VL vision model extracts commodities + weights
- Two bill formats: individual bag weights vs batch totals

- Individual weights shown as EDITABLE inputs (user can fix OCR errors)
- Editing recalculates batch total → commodity total → grand total → amount
- Tesseract.js fallback when server-side OCR unavailable
- Cloudflare Worker proxy bypasses Vercel 10s timeout limit

3. EasyPay Wallet (Coming Soon)

- Full wallet system built but gated behind “Coming Soon” page
- Atomic transfers using MongoDB session transactions
- PayLater credit with TOCTOU protection (\$expr guard)
- IFSC code validation before bank verification
- Safe wallet GET (only returns non-sensitive fields)

4. PWA Features

- Installable on Android (beforeinstallprompt) + iOS (instructions banner)
 - Offline support via Service Worker caching
 - Web App Manifest with maskable icons
 - Custom install banner with role-aware messaging
-

Thought Process: How I Built It

Phase 1: Architecture Decisions

Q: Why Next.js App Router instead of Pages Router? > “App Router gives server components (reduced JS bundle), nested layouts, streaming SSR, and better SEO with metadata API. The home page is a server component — zero client JS shipped for the initial render.”

Q: Why MongoDB instead of PostgreSQL? > “The data is document-oriented — social posts with embedded comments, wallet transactions with flexible metadata, user profiles with role-specific fields. MongoDB’s flexible schema lets us add fields without migrations. Mongoose provides type safety + validation on top.”

Q: Why inline CSS-in-JS instead of Tailwind? > “The home page needs pixel-perfect custom design with animations (gradient shimmer, float, sway). Inline styles give precise control without the class-name overhead. Tailwind is used on some internal pages where utility classes are more efficient.”

Q: Why JWT in localStorage instead of httpOnly cookies only? > “The app needs to work as a PWA offline. localStorage persists the JWT so the user stays logged in even without network. The JWT is also set in an httpOnly cookie for server-side middleware auth. Both are used — cookie for middleware, localStorage for client-side API calls.”

Phase 2: Security Decisions

Q: How did you handle PII (Aadhar numbers)? > “AES-256-GCM encryption via Node.js crypto module. The encryption key is a 64-hex-char env var. Aadhar numbers are encrypted before storing, decrypted on read via Mongoose getters. The model fails closed — if ENCRYPTION_KEY is missing, it throws instead of storing plaintext.”

Q: How did you prevent mass assignment? > “Zod schemas validate every API input. I removed .passthrough() so extra fields are silently dropped. The register route explicitly picks fields for User.create() — even if an attacker sends role: 'admin', Zod rejects it (admin is not in the enum).”

Q: How did you handle the Cloudflare Worker security? > “Originally the worker had hardcoded API credentials (JWT token, chat ID) — I moved them to Cloudflare environment variables. Added origin allowlist (only agrieasy.site), X-Worker-Key authentication header, and imageUrl SSRF validation (only Cloudinary URLs allowed).”

Phase 3: Performance Decisions

Q: How did you optimize the social feed? > “Stale-while-revalidate pattern: on page load, cached posts from localStorage render instantly (0s first paint). Fresh posts are fetched in the background and swap in when ready. This eliminates the loading spinner for returning users.”

Q: How did you handle the Vercel 10s timeout for OCR? > “Vercel Hobby tier kills serverless functions at 10s, but OCR takes 15-25s. I deployed a Cloudflare Worker as a CORS proxy — it has a 30s CPU limit. The browser calls the Worker, the Worker calls the Z-AI vision API, and returns the result with CORS headers.”

Phase 4: Testing & Quality

Q: How did you test the app? > “I wrote a comprehensive test suite using mongodb-memory-server that spins up an in-memory MongoDB instance. 16 test scenarios with 48 assertions covering

registration edge cases (weak passwords, Google users, duplicates, missing fields). I also ran a 104-endpoint API test suite against production — all passed with 0 failures.”

Q: How did you handle the security audit? > “A full security audit found 22 vulnerabilities (5 critical, 6 high, 7 medium, 4 low). I fixed all of them: removed admin role from registration, made PayLater atomic with \$expr guard, added MongoDB session transactions for wallet transfers, fixed interest rate calculation (/100 → /365), and many more.”

PART 2: INTERVIEW QUESTIONS

Basic Full-Stack Questions

Q: What is the difference between server-side rendering and client-side rendering? > SSR: Server generates HTML, sends to browser. Better SEO, faster first paint. CSR: Browser downloads JS, renders. Better interactivity but slower initial load. Next.js App Router uses both — server components for initial render, client components for interactivity.

Q: What is the difference between SQL and NoSQL databases? > SQL: relational, fixed schema, ACID transactions, joins. NoSQL: document-based, flexible schema, horizontal scaling, no joins. I used MongoDB because the data is document-oriented (social posts with embedded comments, flexible user profiles).

Q: What is JWT and how does it work? > JWT = JSON Web Token. Three parts: header (algorithm), payload (userId, email, role, exp), signature (HMAC of header+payload). Server signs with a secret key. Client sends it in Authorization header. Server verifies signature. I use 7-day expiry with tokenVersion for revocation.

Q: What is REST API? > REST = Representational State Transfer. Stateless, HTTP-based, uses methods (GET/POST/PATCH/DELETE). Resources identified by URLs (/api/social/posts/123). I built 72 REST endpoints with consistent response format ({ success, data, meta }).

Q: What is CORS? > Cross-Origin Resource Sharing. Browser security policy blocks requests to different origins. Server sends Access-Control-Allow-Origin header to permit. I configured CORS with origin allowlist (only agrieasy.site) on the Cloudflare Worker.

Q: What is the difference between localStorage and cookies? > localStorage: 5-10MB, no expiry, accessible via JS, sent only in same-origin requests. Cookies: 4KB, has expiry, can be httpOnly (not accessible via JS), sent with every HTTP request. I use both — localStorage for JWT (client-side API calls), httpOnly cookie for middleware auth.

Intermediate Questions

Q: How does your authentication middleware work? > authenticateRequest() extracts JWT from Authorization header or cookie, verifies with jwt.verify() (pinned to HS256 algorithm to prevent algorithm confusion attacks), returns the payload or null. For sensitive endpoints, authenticateRequestAsync() also checks tokenVersion against the database.

Q: How do you handle rate limiting? > In-memory Map-based rate limiter (when no Redis). Each IP/user gets a key with timestamps. On each request, prune expired timestamps, check if count exceeds max. If yes, return 429 with Retry-After header. In production, Upstash Redis provides distributed rate limiting.

Q: What is the stale-while-revalidate pattern? > Show cached data immediately (stale), fetch fresh data in background (revalidate), swap in when ready. I use this for the social feed: localStorage cache renders instantly, fresh posts swap in silently. Also used in the Service Worker for static assets.

Q: How do you validate user input? > Zod schemas on every API endpoint. validateBody(schema, body) returns { success, data, errors }. Password requires uppercase + lowercase + digit + 8+ chars. Phone must be 10-digit Indian mobile. Aadhar must pass Verhoeff checksum. GSTIN must match 15-char format.

Q: How does the PWA service worker work? > Network-first for navigations (always get fresh HTML when online, fall back to cache when offline). Stale-while-revalidate for static assets (serve from cache instantly, fetch new version in background, update cache for next visit). Cache version bumped on each deploy to force-refresh.

Q: What is the TOCTOU vulnerability and how did you fix it? > Time-of-check-to-time-of-use. The PayLater endpoint read wallet balance, checked limit, then wrote — non-atomically. 10 concurrent requests could all pass the check. Fixed with findOneAndUpdate using \$expr: { \$lte: [{ \$add: ['\$paylaterUsed', amount] }, '\$paylaterLimit'] } — MongoDB atomically checks AND writes.

Advanced Questions

Q: How did you implement atomic wallet transfers? >

MongoDB session transactions.

`session.withTransaction(async () => { ... })` wraps the debit + credit + transaction records. If any step fails, the entire transaction rolls back. The debit also has a `$gte` guard (`balance: { $gte: amount }`) as a secondary protection against double-spend.

Q: How does the bill OCR pipeline work? > 1. Client compresses image to 1600px JPEG (85% quality, ~200-400KB) > 2. Base64-encoded → POST `/api/ledger/bill-ocr` > 3. Server sends to Qwen3-VL via OpenRouter with a structured prompt > 4. AI returns JSON: `{ commodities: [{ name, batches: [{ bagCount, weight, individualWeights[] }] }] }` > 5. Client displays individual weights as editable inputs > 6. User can fix OCR errors → recalculates totals in real-time > 7. Fallback: Tesseract.js (client-side, free, offline) if server OCR fails

Q: How did you handle the social feed ranking algorithm? >

$\text{rankScore} = (\text{likes} \times 5 + \text{comments} \times 8 + \text{saves} \times 6 + \text{shares} \times 10) \times \text{recency_decay}$. Recency decays over 1 week ($1 - \text{ageHours}/168$). The score is recalculated on every save via a Mongoose pre-save hook. The feed fetches 60% ranked + 40% latest and interleaves them — so users see both popular and fresh content.

Q: How do you handle error responses consistently? >

Custom `apiSuccess()` and `apiError()` helpers. Error codes: AUTH_REQUIRED (401), FORBIDDEN (403), VALIDATION_ERROR (400), NOT_FOUND (404), RATE_LIMITED (429), INTERNAL_ERROR (500). In production, error messages are sanitized to remove file paths and stack traces.

Q: How did you handle the view tracking for KrishiClips? >

Each clip has `viewedBy: [ObjectId]` and `viewedByIpHash: [String]`. When a clip becomes active in the viewport, the client fires POST `/api/social/clips/[id]/view`. The server uses `$addToSet` (idempotent — same user calling 100 times = 1 view). The views counter only increments when a NEW unique viewer is detected.

Q: What is the difference between findOneAndUpdate and updateOne? >

`findOneAndUpdate` returns the updated document (with `{ new: true }`). `updateOne` only returns the write result (matched count, modified count). I use `findOneAndUpdate` when I need the updated value (e.g., wallet balance after transfer) and `updateOne` when I just need to know if the update succeeded.

System Design Questions

Q: How would you scale this app to 1 million users? > 1. MongoDB sharding (shard by userId or region) > 2. Redis for rate limiting + session caching (already have Upstash integration) > 3. CDN for static assets (Vercel Edge already does this) > 4. Read replicas for MongoDB (social feed reads >> writes) > 5. Queue-based OCR processing (BullMQ + Redis) instead of synchronous API > 6. Image optimization pipeline (Cloudinary transformations) > 7. Horizontal scaling of API routes (Vercel auto-scales serverless functions)

Q: How would you design the notification system? > Currently using MongoDB polling. For scale: WebSocket (Socket.io) for real-time delivery. Store notifications in MongoDB with a TTL index (auto-expire after 30 days). Push notifications via Firebase Cloud Messaging for PWA. I already have the Notification model + read/unread tracking.

Behavioral / Project-Specific Questions

Q: What was the hardest bug you fixed? > The social feed was returning 500 errors because a post had a corrupt userId reference (pointing to a deleted user). Mongoose's .populate() threw an error that crashed the entire endpoint. I fixed it with a safePopulate() helper that catches populate errors per-post and returns userId: null — the frontend renders “Unknown User” instead of crashing.

Q: What would you do differently? > 1. Use a proper state management library (Zustand) instead of prop drilling > 2. Add integration tests with Playwright for E2E flows > 3. Use Redis from day 1 for rate limiting + caching > 4. Implement GraphQL for the social feed (over-fetching with REST) > 5. Add Sentry for production error monitoring

Q: How do you handle deployment? > Git push to main → Vercel auto-deploys (build + preview + production). Cloudflare Workers Builds auto-deploys the OCR worker. I use git cherry-pick workflow for clean commits on the rebased branch. Environment variables set on Vercel dashboard (MONGODB_URI, JWT_SECRET, ENCRYPTION_KEY, etc.).

Q: What is your CI/CD pipeline? > Vercel handles the CD (auto-deploy on push). Cloudflare Workers Builds deploys the OCR worker. No GitHub Actions — Vercel's built-in CI runs next build and deploys if it passes. I run npx tsc --noEmit + npx eslint --quiet locally before pushing.

PART 3: QUICK REFERENCE

Numbers to Remember

- **72 API endpoints** (44 GET, 51 POST, 9 PATCH, 6 DELETE)
- **55 pages** (48 static, 9 dynamic)
- **18 Mongoose models**
- **~28,700 lines** of TypeScript
- **3 user roles** (Farmer, Buyer, Transporter)
- **22 security vulnerabilities** found + fixed in audit
- **104 endpoints tested** — all pass
- **48 unit test assertions** — all pass
- **PWA** — installable, offline support, service worker v25

Key Technologies to Mention

- Next.js 16 App Router (server components, streaming SSR)
- TypeScript strict mode
- MongoDB + Mongoose 9 (18 models, aggregation pipelines)
- JWT auth with tokenVersion revocation
- Web Audio API (programmatic sound generation)
- Cloudflare Workers (CORS proxy for OCR)
- PWA (service worker, manifest, install prompt)
- AES-256-GCM encryption for PII
- MongoDB session transactions for atomicity
- Zod validation on every endpoint
- Stale-while-revalidate caching pattern
- \$addToSet for idempotent view tracking
- \$expr guard for TOCTOU prevention

If Asked “Tell me about your project”

“AgriEasy is a three-sided agricultural marketplace PPA connecting Farmers, Buyers, and Transporters. The frontend is Next.js 16 with TypeScript, the backend is 72 REST API endpoints with MongoDB, and it’s deployed on Vercel as a PWA with offline support.

The app handles the entire supply chain: farmers list their produce, buyers search and place orders, transporters deliver with live tracking, and payments settle through the wallet system. I also built an Instagram-style social feed with KrishiClips, a bill calculator with AI OCR, and a ledger for tracking earnings and expenses.

The most interesting technical challenges were: making wallet transfers atomic with MongoDB session transactions, preventing TOCTOU race conditions in the PayLater credit system using \$expr guards, implementing stale-while-revalidate caching for 0-second feed loads, and building a Cloudflare Worker proxy to bypass Vercel's 10-second timeout for OCR processing.

I also ran a full security audit that found 22 vulnerabilities — 5 critical, including hardcoded API credentials, admin role registration, and race conditions in financial endpoints. I fixed all of them.”